
SPECIFICATION

Calypso Terminal API Reader

Version 3.0.0-SNAPSHOT, 2026-07-20



Warning. The present document cancels and replaces the previous release.

Link and Contact



Website
<https://calypsonet.org>



Support
support@calypsonet.org

Copyright © Calypso Networks Association, <https://calypsonet.org>.

*This specification is licensed under the **Creative Commons Attribution-NoDerivatives 4.0 International** (CC BY-ND 4.0).*

*You are free to **share**—copy and redistribute this document in any medium or format for any purpose, even commercially—under the following terms:*

- **Attribution**—You *MUST* give appropriate credit to the Calypso Networks Association, provide a link to the license, and indicate if changes were made. You *MAY* do so in any reasonable manner, but not in any way that suggests the Calypso Networks Association endorses you or your use.
- **NoDerivatives**—If you remix, transform, or build upon this document, you *MAY NOT* distribute the modified material.

*No additional restrictions—You **MAY NOT** apply legal terms or technological measures that legally restrict others from doing anything the license permits.*

Implementation grant—The **NoDerivatives** condition above applies to this specification **document** only—its text, tables and diagrams. It does **not** restrict the use of the technical information the document contains. The Calypso Networks Association hereby grants everyone an irrevocable, worldwide, royalty-free permission to use the information contained in this specification to design, develop, and distribute software implementations of this API, in any programming language, under the license of their choice.

The full license text is available at <https://creativecommons.org/licenses/by-nd/4.0/>.

Table of Contents

1. Abstract	6
2. Introduction	7
2.1. Purpose	7
2.2. Scope	7
2.3. Conformance	7
2.4. Document Conventions	8
2.4.1. Naming	8
2.4.2. Data types	8
2.4.3. Type tables	8
2.4.4. Operation tables	9
2.4.5. Operation contracts	9
2.4.6. Concurrency	9
2.5. References and Resources	10
2.5.1. Calypso References	10
2.5.2. Normative References	10
2.5.3. External Resources	10
3. Glossary and Acronyms	11
3.1. Glossary	11
3.2. Acronyms	11
4. Architectural Overview	13
4.1. Functional positioning	13
4.2. Logical namespaces	13
4.3. Multi-channel design	14
4.4. Card selection modes	14
4.5. Reader observation lifecycle	15
4.6. UML class diagram	17
5. API Specification	18
5.1. Constants	18
5.1.1. Reader API Properties	18
5.2. Data	18
5.2.1. Basic Card Selector	18
5.2.2. Card Detection Settings	18
5.2.3. Card Reader Event	19
5.2.4. ISO Card Selector	19
5.2.5. Multichannel Card Selection Result	20
5.2.6. Sequential Card Selection Result	21
5.2.7. Single Card Selection Result	21
5.3. API Interfaces	22
5.3.1. Card Reader	22
Get Name	22
Is Card Present	22
Is Contactless	22
5.3.2. Card Reader Provider	22
Find Reader	23
Get Reader	23
Get Reader Names	23
Get Readers	23

5.3.3. Card Selection Manager	24
Export Card Selection Scenario	24
Export Processed Card Selection Scenario	25
Import Card Selection Scenario	25
Prepare Selection Case	25
5.3.4. Card Selector	26
5.3.5. Multichannel Card Selection Manager	26
Import Processed Card Selection Scenario	26
Process Card Selection Scenario	27
5.3.6. Observable Card Reader	27
Clear Scheduled Card Selection Scenario	28
End Card Processing	28
Start Card Detection	28
Stop Card Detection	29
5.3.7. Reader API Factory	29
Create Multichannel Card Selection Manager	29
Create Sequential Card Selection Manager	29
Create Single Card Selection Manager	29
Get Card Reader Provider	30
5.3.8. Scheduled Card Selections Response	30
5.3.9. Sequential Card Selection Manager	30
Import Processed Card Selection Scenario	31
Parse Scheduled Card Selections Response	31
Process Card Selection Scenario	31
Schedule Card Selection Scenario	32
5.3.10. Single Card Selection Manager	32
Import Processed Card Selection Scenario	32
Parse Scheduled Card Selections Response	33
Process Card Selection Scenario	33
Schedule Card Selection Scenario	33
5.4. SPI Interfaces	34
5.4.1. Card Reader Event Handler	34
On Reader Error	34
On Reader Event	34
5.4.2. Card Selection Extension	34
5.4.3. Card Transaction Manager	35
Process Commands	35
5.4.4. ISO Card Transaction Manager	35
As Multichannel Card Transaction Manager	35
5.4.5. ISO Smart Card	36
Get Select Application Response	36
Is Basic Channel	36
5.4.6. Multichannel Card Transaction Manager	36
Close Channel	37
Process Commands And Close Channel	37
5.4.7. Smart Card	37
Get Power On Data	37
Is Active	38
5.5. Enumerations	38

5.5.1. Card Presence Notification Policy..... 38

5.5.2. Card Reader Event Type..... 39

5.5.3. Channel Selection Policy 39

5.5.4. Detection Mode 39

5.5.5. File Control Information..... 40

5.5.6. File Occurrence..... 40

5.6. Errors 40

5.6.1. Card Communication 40

5.6.2. Invalid Card Response 41

5.6.3. Reader Communication 41

1. Abstract

This document is the official technical specification of the **Terminal Reader API** standardised by the **Calypso Networks Association** (CNA). It defines, in a language-agnostic way (using a notation inspired by the Kotlin language), the interfaces, classes, enumerations, errors, structural relationships and behavioural requirements that any conforming implementation of the Terminal Reader API MUST satisfy.

The Terminal Reader API is part of the broader CNA **Terminal API** family and acts as the contract through which terminal applications discover card readers, observe card events, select cards and exchange data with them.

Document Status

Reference	YYMMDD-SP-CNATerminalAPI-Reader
Short name	CNA-TR-API
Version	3.0.0-SNAPSHOT
Revision date	2026-07-20
Editor	Calypso Networks Association
Source repository	https://github.com/calypsonet/calypsonet-terminal-reader-uml-api
Reference license	Creative Commons Attribution-NoDerivatives 4.0 International (CC BY-ND 4.0)



The present document specifies the 3.0.0-SNAPSHOT of the Terminal Reader API. It defines a single, coherent baseline against which conforming implementations are evaluated; the **Since** column indicates the version in which each member was introduced.

Revision List



This section lists the high-level changes per version. The complete, fine-grained changelog is maintained in the **CHANGELOG.md** file at the root of the source repository.

Version / Date	Modifications
v3.0.0-SNAPSHOT 2026-07-20	Baseline.

2. Introduction

2.1. Purpose

The Terminal Reader API defines a standardised set of contracts allowing terminal applications to interact with card readers and the cards presented to them, independently of:

- the underlying reader hardware (contact, contactless, virtual, etc.),
- the card technology family ([ISO/IEC 7816-4](#), [ISO/IEC 14443](#), proprietary),
- the implementation technology of the application (programming language, runtime).

Conforming implementations of this specification **MUST** allow extension modules (e.g. card-specific APIs such as the Calypso Card API) to plug in transparently through the Service Provider Interface (SPI) mechanism defined in `reader.selection.spi` and `reader.transaction.spi`.

2.2. Scope

This specification covers:

- the discovery and basic interrogation of card readers,
- the observation of reader events related to card presence and selection,
- the configuration of card detection (RF technologies, detection mode, ECP frame),
- the description and orchestration of card selection scenarios, including multi-channel scenarios,
- the abstraction of the physical and logical communication channels established with cards,
- the contract surface required by card extension modules to interoperate with reader implementations,
- the management of single-channel and multi-channel card transactions.

The following topics are **out of scope**:

- the wire-level protocols used between reader and card (e.g. APDU framing, ISO/IEC 14443 anti-collision),
- the cryptographic operations performed by or on behalf of the card,
- the lifecycle and packaging of reader drivers,
- the user interface presented by the terminal application.

2.3. Conformance

A software product conforms to this specification if and only if:

1. it implements every interface and class defined in [Chapter 5](#) with the operations and semantics prescribed in this document,
2. it raises errors exclusively of the types defined in [Section 5.6](#) for the situations described,
3. it preserves the structural relationships described in [Chapter 4](#).

Throughout this specification, the key words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY** and **OPTIONAL** are to be interpreted as described in [RFC 2119](#) and [RFC 8174](#) when, and only when, they appear in all capitals.

In conformance with [RFC 2119](#), **SHALL** is equivalent to **MUST**; this specification uses **MUST** exclusively in order

to avoid ambiguity.

2.4. Document Conventions

2.4.1. Naming

Type names follow **UpperCamelCase**; operation, parameter and enumeration-member names follow **lowerCamelCase** except for enumeration values which use **UPPER_SNAKE_CASE**. Names defined by this specification are reproduced as-is in the UML class diagram ([Section 4.6](#)).

2.4.2. Data types

The following type names, whose notation is inspired by the Kotlin language, are used in operation signatures and are mapped, in each binding, to the closest equivalent native type. Implementations **MUST** preserve the semantic intent rather than the syntactic form.

Type	Description	Typical bindings
String	Sequence of characters, UTF-8 encoded by default.	String , string , str .
Boolean	Two-state truth value.	boolean , bool .
Int	Signed 32-bit integer.	int , Int32 , i32 .
Long	Signed 64-bit integer.	long , Int64 , i64 .
ByteArray	Ordered sequence of octets.	byte[] , [u8] , bytes .
List<T>	Ordered collection of T values, may contain duplicates.	List<T> , Vec<T> .
Set<T>	Unordered collection of unique T values.	Set<T> , HashSet<T> .
Map<K, V>	Associative array binding keys of type K to values of type V .	Map<K, V> , Dictionary<K, V> , HashMap<K, V> .
Any	Value of any type, root of the type hierarchy.	Object , Any , object , Box<dyn Any> .
T?	Nullable value: either a T value or null .	Optional<T> , T? , Option<T> .
null	Absence of value, the alternative held by a nullable type.	null , nil , None .
Self	Own type of the receiver: a fluent operation returns the instance it was called on.	Self , T (recursive generic).
Unit	Indicates that an operation returns no value.	void , Unit , () .

2.4.3. Type tables

Each interface, class, enumeration and error defined in this document is described by a table of the form:

Name	The type's normalized name.
Kind	The category of the type (interface, class, enumeration, error, etc.).
Namespace	The namespace in which the type is defined.
Extends	The type extended by this type, when applicable.
Implements	The interface implemented by this type, when applicable.
Since	The version of the API in which the type was introduced.
Purpose	A normative description of the type's responsibility.

See also

Cross-references to related operations or types, each a hyperlink to the corresponding table. Present only when applicable.

The **Purpose** row states, in one or two sentences, the responsibility of the type and the way an instance is obtained. It is meant to be scanned, not read: any content that requires structure or depth—rules, lists, notes, value tables or examples—is placed as prose below the table.

2.4.4. Operation tables

Each operation defined in this document is described by a table of the form:

Signature	The operation's signature.
Since	The version of the API in which the operation was introduced.
Description	A normative description of the operation's behaviour.
Parameters	A description of each input parameter.
Returns	A description of the returned value, if any.
Pre-conditions	The conditions the caller MUST satisfy before invoking the operation, each prefixed by its nature.
Errors	A list of error conditions a correct caller must handle, each mapped to a specific error type.
See also	Cross-references to related operations or types, each a hyperlink to the corresponding table. Present only when applicable.

2.4.5. Operation contracts

To avoid repetition, the following rules apply to every operation table in [Chapter 5](#) and are therefore not restated for each operation:

- **Pre-conditions.** The **Pre-conditions** row states what the caller **MUST** guarantee before invoking the operation. Violating one is a contract error, never listed in the **Errors** row, which describes only the outcomes a correct caller must handle. Each entry is prefixed by its nature, which determines the error raised when the condition is not met:
 - **Argument**—an illegal argument error. Applies to any invalid argument, including values bounded by a card or SAM protocol.
 - **Range**—an index out of bounds error. Applies when the argument designates a position within a collection or memory image exposed by the API.
 - **State**—an illegal state error.
 - **Capability**—an unsupported operation error.
- **Non-null results.** Unless the return type is marked nullable (**T?**) or the **Returns** row states otherwise, an operation returns a non-**null** value.
- **Collections.** Unless the return type is marked nullable (**T?**), an operation whose return type is a collection (e.g. **List**, **Set**, **Map**) returns an empty collection rather than **null**.
- **Fluent composition.** Builder-style operations return the current instance so that calls can be chained.

2.4.6. Concurrency

Unless explicitly stated otherwise, the stateful types defined by this specification are **not** thread-safe. A given instance **MUST** be accessed from a single thread at a time; concurrent use of the same instance without external synchronisation results in undefined behaviour. Distinct instances **MAY** be used concurrently.

2.5. References and Resources

2.5.1. Calypso References

References to CNA Terminal APIs designate the indicated **major** version; any later release within the same major is backward-compatible per that API's evolution policy.

Reference	Document
[CNA-TRD-API]	CNA Terminal API—Reader Definitions (SP-CNATerminalAPI-ReaderDefinitions), version 1.0, Calypso Networks Association. Provides the RfTechnology and CardType enumerations referenced by this specification.

2.5.2. Normative References

Reference	Document
[RFC 2119]	Key words for use in RFCs to Indicate Requirement Levels , S. Bradner, March 1997.
[RFC 8174]	Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words , B. Leiba, May 2017.
[ISO/IEC 7816]	Identification cards—Integrated circuit cards.
[ISO/IEC 14443]	Identification cards—Contactless integrated circuit cards—Proximity cards.

2.5.3. External Resources

Resource	Link
Calypso Networks Association	https://calypsonet.org
Terminal APIs website	https://terminal-api.calypsonet.org
Terminal APIs documentation	https://docs.terminal-api.calypsonet.org

3. Glossary and Acronyms

3.1. Glossary

Term	Definition
Card	Any physical or virtual smart card that can be presented to a reader for selection and dialog.
Card Reader	Hardware or software component capable of detecting cards and exchanging data with them.
Card Selection	Sequence of operations used to identify, filter and prepare a card for application-level dialog.
Logical Channel	Software-level communication path established with a card after a successful selection.
Basic Channel	The default logical channel (channel 0) of an ISO/IEC 7816-4 card.
Multi-Channel	Capability of an ISO/IEC 7816-4 card to host several active logical channels in parallel.
Physical Channel	Hardware-level communication path between the reader and the card.
Observable Reader	Reader implementation able to notify a registered event handler asynchronously of card-related events.
Power-on Data	Data returned by the reader at the moment a card is detected (e.g. ATR, virtual ATR, anti-collision data).
Card Application	Application Dedicated File (DF) selected on a card, identified by its AID, as defined in ISO/IEC 7816-4.
AID	Application IDentifier as defined in ISO/IEC 7816-4 (5 to 16 bytes).
APDU	Application Protocol Data Unit, the message format used between the terminal and the card, as defined in ISO/IEC 7816-3.
ECP	Enhanced Contactless Polling, an extended polling mechanism enabling fast card detection in mobile transit deployments.
RF Technology	Radio-frequency communication technology used by a contactless reader to detect a card, as defined in ISO/IEC 14443.
Card Type	Identified product or transport-protocol family of a detected card.
SPI	Service Provider Interface—a set of interfaces designed to be implemented by an extension module.

3.2. Acronyms

Abbreviation	Expansion
AID	Application IDentifier
APDU	Application Protocol Data Unit
ATR	Answer To Reset
CNA	Calypso Networks Association
DF	Dedicated File
ECP	Enhanced Contactless Polling
FCI	File Control Information
FCP	File Control Parameters
FMD	File Management Data
ISO	International Organization for Standardization
RF	Radio Frequency
RFC	Request For Comments

Abbreviation	Expansion
SPI	Service Provider Interface
UML	Unified Modelling Language

4. Architectural Overview

4.1. Functional positioning

The Terminal Reader API sits between the terminal application and the reader drivers, exposing a stable contract on each side:

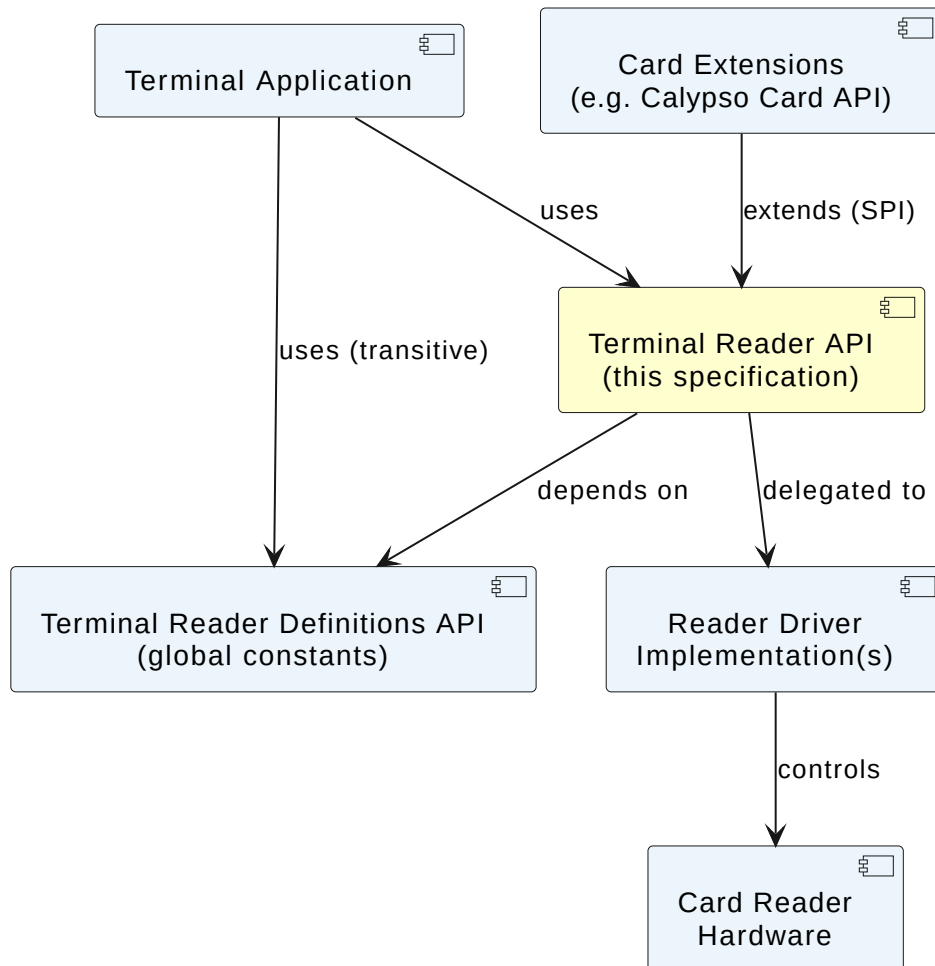


Figure 1. Terminal Reader API—Architecture Overview

4.2. Logical namespaces

Namespace	Role	Summary
<code>reader</code>	Public API	Entry points of the API and the abstractions used to manipulate readers and the events they produce; exposes CardReaderProvider to discover and access the readers available in the execution environment.
<code>reader.spi</code>	SPI	Contracts implemented by the application (as opposed to the reader implementation) when readers are observed.
<code>reader.selection</code>	Public API	Orchestration of card selection scenarios: the hierarchy of card selection managers (one per selection mode), the family of selectors, the channel and notification policies and the structures exposing selection results.

Namespace	Role	Summary
<code>reader.selection.spi</code>	SPI	Contracts implemented by card extensions to plug into the selection machinery.
<code>reader.transaction.spi</code>	SPI	Common contract shared by all card transaction managers exposed by card extensions; its three interfaces form a hierarchy letting each extension anchor its transaction manager at the capability level matching its card model (non-ISO, optionally multi-channel, intrinsically multi-channel).

4.3. Multi-channel design

The Terminal Reader API explicitly supports ISO/IEC 7816-4 cards that host several active logical channels in parallel. The design follows a three-tier hierarchy in the `reader.transaction.spi` namespace:

1. `CardTransactionManager`—root contract common to every transaction manager, regardless of channel model.
2. `IsoCardTransactionManager`—intermediate contract for ISO/IEC 7816-4 cards for which multi-channel is an **optional capability** of the underlying card. It exposes the cast operation `asMultichannelCardTransactionManager` that returns a `MultichannelCardTransactionManager` when the card supports it.
3. `MultichannelCardTransactionManager`—contract for cards that are **intrinsically multi-channel** by design. Consumer APIs whose target cards are always multi-channel SHOULD make their transaction manager extend this contract directly.

4.4. Card selection modes

A card selection scenario—prepared through one of the card selection managers described in `CardSelectionManager`, whose type determines the selection mode—can be operated in two ways, chosen according to the reader capability and the use case.



This section is *informative*. It gives an end-to-end view of the two modes; the normative behaviour of each operation is defined in the sections of the card selection managers.

Synchronous (explicit)

The card is already present on a `CardReader`. The application runs the scenario itself with the `processCardSelectionScenario` operation of its manager and obtains the corresponding result immediately (`SingleCardSelectionResult`, `SequentialCardSelectionResult` or `MultichannelCardSelectionResult`). The **same entity** manages both the selection and the subsequent card processing.

Asynchronous (scheduled / event-driven)

On an `ObservableCardReader`, the scenario is registered ahead of time with `scheduleCardSelectionScenario`—available on the single-channel managers `SingleCardSelectionManager` and `SequentialCardSelectionManager` only—and executed automatically by the reader upon card insertion. The outcome is delivered as a `CardReaderEvent` to the `CardReaderEventHandler`—its type (`CARD_MATCHED` or `CARD_INSERTED`) depending on the `CardPresenceNotificationPolicy`—and decoded with the `parseScheduledCardSelectionsResponse` operation of the same manager. Here the **reader** drives the selection while the **event handler** performs the card processing.

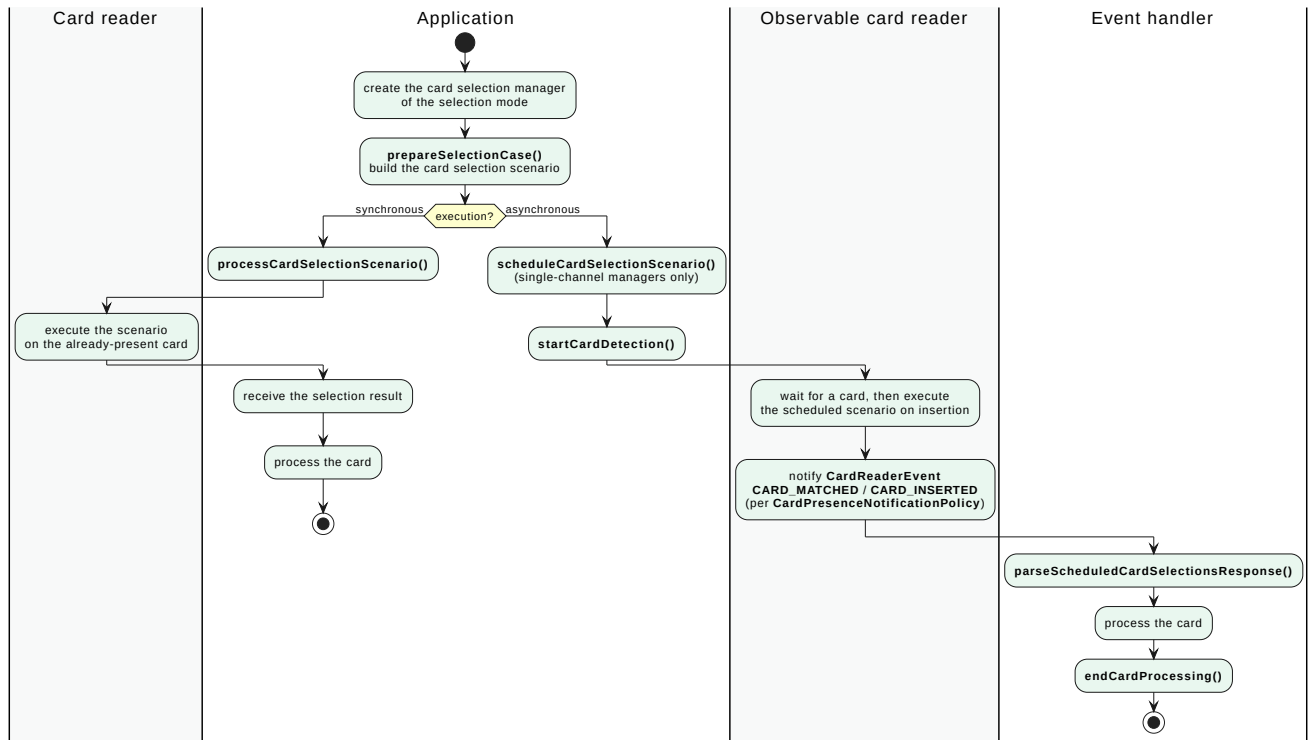


Figure 2. Terminal Reader API—Card Selection Modes Activity Diagram

The runtime execution of the asynchronous mode—the reader states and transitions—is detailed in [Reader observation lifecycle](#).

4.5. Reader observation lifecycle

An **ObservableCardReader** tracks card insertion and removal through an internal state machine. The diagram below gives the integrator a complete view of that lifecycle: how **startCardDetection** / **stopCardDetection** and **endCardProcessing** move the reader between states, when a **CardReaderEvent** is produced, and how the **DetectionMode** and **CardPresenceNotificationPolicy** settings steer the flow.



This section is *informative*. The state and trigger names shown are internal to the implementation and are not part of the normative API surface; they are provided only to explain the observed runtime behavior.

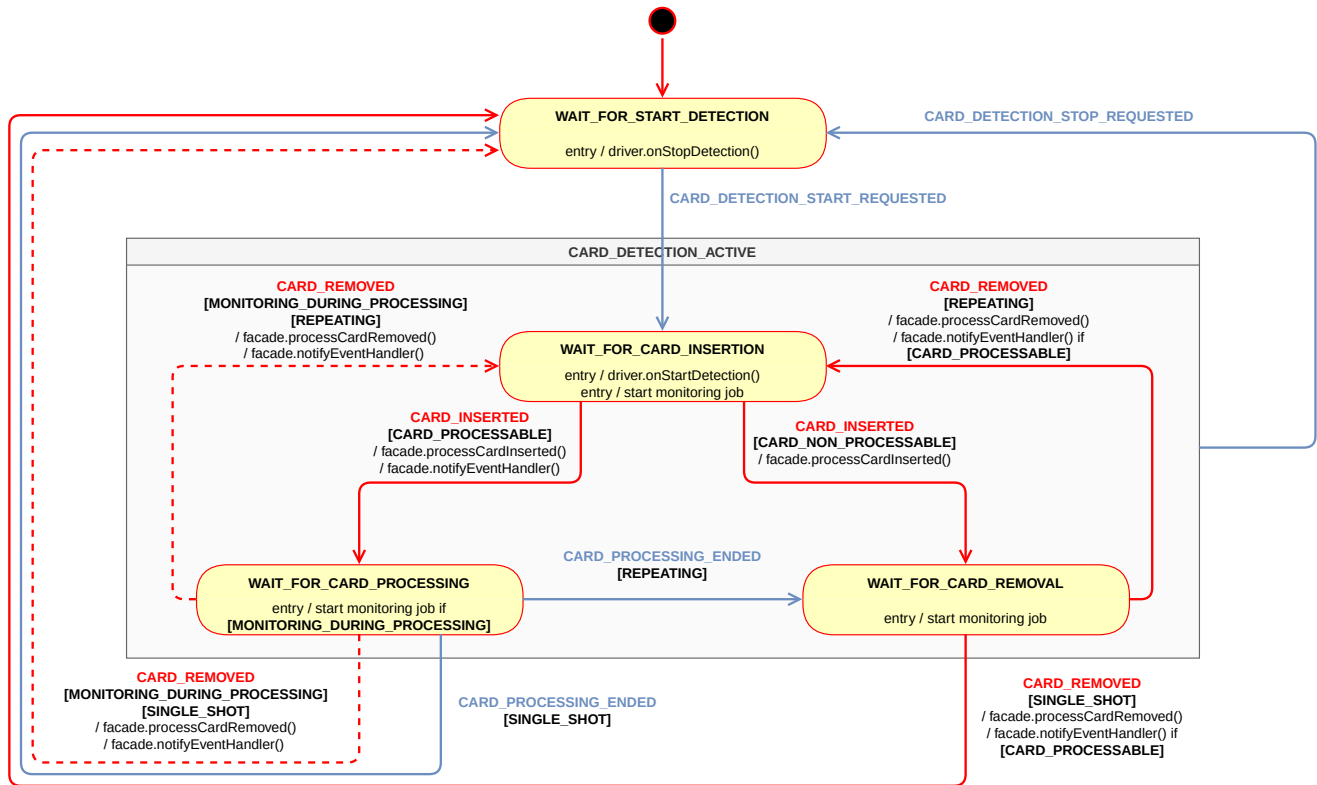


Figure 3. Card Reader Observation — FSM State Diagram

The diagram uses the following notation:

Triggers—events that drive the transitions:

- **Application** triggers (blue transitions in the diagram):
 - **CARD_DETECTION_START_REQUESTED**—the application called startCardDetection.
 - **CARD_DETECTION_STOP_REQUESTED**—the application called stopCardDetection (factorized on the composite-state boundary).
 - **CARD_PROCESSING_ENDED**—the application called endCardProcessing.
- **Hardware** triggers (red transitions in the diagram):
 - **CARD_INSERTED**—the monitoring job detected a card.
 - **CARD_REMOVED**—the monitoring job detected card removal.

Reader capabilities:

- **[MONITORING_DURING_PROCESSING]**—the reader can detect card removal while processing is running, **AND** card processing is executed in a dedicated thread.

Card eligibility:

- **[CARD_PROCESSABLE]**—the inserted card matched at least one case of the card selection scenario, **OR** the notification policy covers every detected card regardless of selection (**CardPresenceNotificationPolicy == ALWAYS**).
- **[CARD_NON_PROCESSABLE]**—the inserted card matched no case of the card selection scenario, **AND** the notification policy is restricted to matched cards (**CardPresenceNotificationPolicy == MATCHED_ONLY**).

Detection mode—the active DetectionMode, evaluated when a card-processing cycle ends:

- [REPEATING]—detection resumes after each card-processing cycle, ready for the next card.
- [SINGLE_SHOT]—detection stops after the first card-processing cycle; a new call to startCardDetection is required to detect further cards.

Actions—effect executed on a transition:

- / `facade.operation()`—a public API call (event production or event-handler notification).
- / `driver.operation()`—a low-level hardware call (hardware notification).
- / `start job`—a monitoring job submitted to the background executor.

4.6. UML class diagram

The following UML class diagram provides a visual representation of the types and relationships defined by this specification:

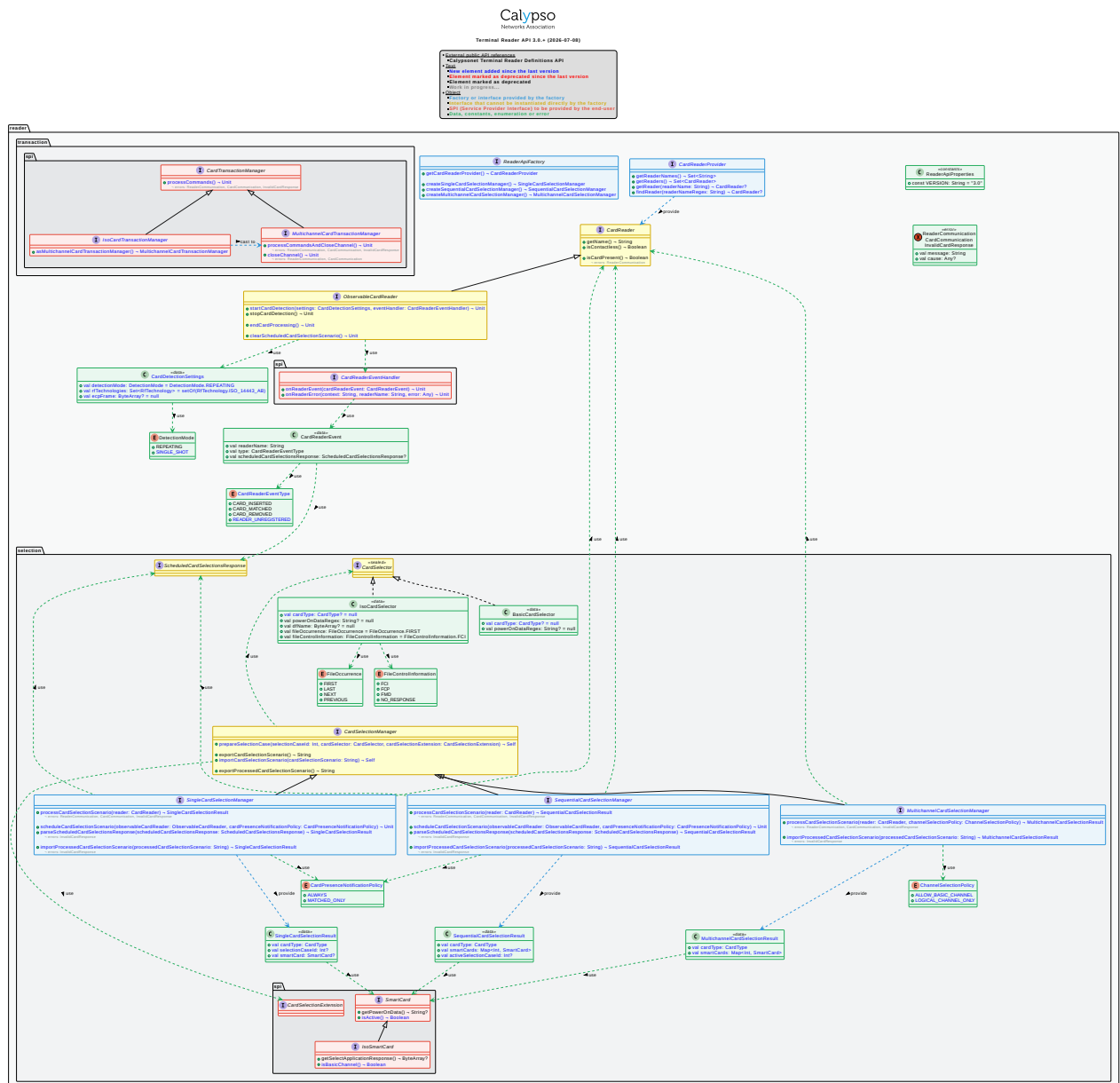


Figure 4. Terminal Reader API—UML Class Diagram

5. API Specification

5.1. Constants

5.1.1. Reader API Properties

Name	ReaderApiProperties
Kind	Constants
Namespace	reader
Since	1.0
Purpose	Exposes the immutable properties of the Terminal Reader API.

Name	Type	Since	Description
VERSION	String	1.0	String representation of the version of the API implemented by the conforming binding. The value is a dotted decimal of the form MAJOR.MINOR; it is "3.0" for this revision of the specification.

5.2. Data

5.2.1. Basic Card Selector

Name	BasicCardSelector
Kind	Data
Namespace	reader.selection
Implements	<u>CardSelector</u>
Since	2.0
Purpose	Basic, technology-agnostic card selector, built directly by the application.

BasicCardSelector carries no ISO/IEC 7816-4-specific filter; it exists to express their absence at the type level. Every filter is **optional** and MAY be combined with the others. When no filter is set, any card that responds when presented to the reader is considered selected; when one or more filters are set, the card is rejected as soon as one of them does not match.

Name	Type	Default	Since	Description
cardType	CardType? (CNA-TRD-API)	null	3.0	Restricts the selection to cards whose detected type matches this value. The value UNKNOWN MAY be used to capture cards whose type could not be identified. null disables the filter.
powerOnDataRegex	String?	null	2.0	Restricts the selection to cards whose power-on data, as provided by the reader, matches this regular expression. null disables the filter.

5.2.2. Card Detection Settings

Name	CardDetectionSettings
Kind	Data
Namespace	reader

Since	3.0
Purpose	Carries the configuration of a card detection cycle, built directly by the application.

Name	Type	Default	Since	Description
detectionMode	<u>DetectionMode</u>	DetectionMode.REPEATING	3.0	Governs the reader behaviour once a card-processing cycle has terminated. With REPEATING , the reader automatically resumes polling and waits for the next card insertion, enabling continuous, unattended operation. With SINGLE_SHOT , the reader stops detection after the current cycle; a new call to <u>startCardDetection</u> is required before another card can be detected.
rfTechnologies	Set<RfTechnology> (CNA-TRD-API)	setOf(RfTechnology.ISO_14443_A_B)	3.0	The set of RF technologies the reader activates during polling. Not relevant for contact readers, where it is accepted without effect. MUST NOT be empty.
ecpFrame	ByteArray?	null	3.0	The ECP (Enhanced Contactless Polling) frame to emit at polling startup, treated by this specification as opaque binary data constructed by the application in accordance with the relevant ECP specification and transmitted as-is to the reader. Relevant only on readers supporting ECP; on the others the frame is silently ignored and a WARN -level message is typically logged. null means that no ECP frame is emitted. When set, it MUST NOT be empty.

5.2.3. Card Reader Event

Name	CardReaderEvent
Kind	Data
Namespace	reader
Since	1.0
Purpose	Data container describing a change of state observed by a CardReader .

A **CardReaderEvent** carries the origin of the event (the reader name), the event type and, when available, the card selection response. The circumstances under which each event type is emitted are described in [Reader observation lifecycle](#).

Name	Type	Default	Since	Description
readerName	String	—	1.0	Name of the reader that generated the event. Non-empty.
type	<u>CardReaderEventType</u>	—	1.0	Type of the event.
scheduledCardSelectionResponse	<u>ScheduledCardSelectionResponse?</u>	—	1.0	Response of the selection scenario scheduled on the reader. It is null only when no selection scenario has been scheduled; in every other case it is non- null . When non- null , it MUST be passed for interpretation to the parseScheduledCardSelectionsResponse operation of the card selection manager that scheduled the associated selection scenario, or of another manager of the same type sharing the same context—that is, one that has previously prepared the same selection scenario.

5.2.4. ISO Card Selector

Name	IsoCardSelector
Kind	Data
Namespace	reader.selection

Implements	<code>CardSelector</code>
Since	2.0
Purpose	ISO/IEC 7816-4 card selector, built directly by the application.

`IsoCardSelector` adds the ISO/IEC 7816-4-specific filters to those common to every selector. They are all **optional** and MAY be combined. When no filter is set, any card that responds when presented to the reader is considered selected; when one or more filters are set, the card is rejected as soon as one of them does not match.

Name	Type	Default	Since	Description
<code>cardType</code>	<code>CardType?</code> (CNA-TRD-API)	<code>null</code>	3.0	Restricts the selection to cards whose detected type matches this value. The value UNKNOWN MAY be used to capture cards whose type could not be identified. <code>null</code> disables the filter.
<code>powerOnDataRegex</code>	<code>String?</code>	<code>null</code>	2.0	Restricts the selection to cards whose power-on data, as provided by the reader, matches this regular expression. <code>null</code> disables the filter.
<code>dfName</code>	<code>ByteArray?</code>	<code>null</code>	2.0	Selects a card application DF by its name: the DF is selected only if its name starts with this AID, as defined by ISO/IEC 7816-4 section 4.2. The value is used as a parameter of the Select Application ISO card command and MUST contain 5 to 16 bytes. <code>null</code> disables the filter.
<code>fileOccurrence</code>	<code>FileOccurrence</code>	<code>FileOccurrence.FIRST</code>	2.0	File occurrence mode (see ISO/IEC 7816-4).
<code>fileControlInformation</code>	<code>FileControlInformation</code>	<code>FileControlInformation.FCI</code>	2.0	File control mode (see ISO/IEC 7816-4).

5.2.5. Multichannel Card Selection Result

Name	<code>MultichannelCardSelectionResult</code>
Kind	Data
Namespace	<code>reader.selection</code>
Since	3.0
Purpose	Result of a selection scenario executed by a <code>MultichannelCardSelectionManager</code> .

Each successful selection case has been placed on its own logical channel. All the smart cards exposed by the result are active in parallel; the one placed on the basic channel, if any, is identified through `IsoSmartCard.isBasicChannel`.

Name	Type	Default	Since	Description
<code>cardType</code>	<code>CardType</code> (CNA-TRD-API)	—	3.0	Type of the detected card, as defined by the <code>CardType</code> enumeration of the Terminal Reader Definitions API (see CNA-TRD-API). UNKNOWN when the card type could not be identified. The value is intrinsically tied to the lifecycle of this result.
<code>smartCards</code>	<code>Map<Int, SmartCard></code>	—	3.0	All <code>SmartCard</code> instances corresponding to successful selection cases, each active on its own logical channel, indexed by the <code>selectionCaseId</code> supplied to <code>prepareSelectionCase</code> . Possibly empty.

5.2.6. Sequential Card Selection Result

Name	SequentialCardSelectionResult
Kind	Data
Namespace	reader.selection
Since	3.0
Purpose	Result of a selection scenario executed by a <u>SequentialCardSelectionManager</u> .

Every selection case of the scenario has been tried. The result exposes the smart cards of all the successful selection cases, indexed by their **selectionCaseId**. Since the card is operated on a single channel, only the smart card of the last successful selection case, in execution order, remains active; the others are deactivated but still expose the data collected during their selection (see [the SmartCard lifecycle](#)). **activeSelectionCaseId** is **null** if and only if **smartCards** is empty.

Name	Type	Default	Since	Description
cardType	CardType (CNA-TRD-API)	—	3.0	Type of the detected card, as defined by the CardType enumeration of the Terminal Reader Definitions API (see CNA-TRD-API). UNKNOWN when the card type could not be identified. The value is intrinsically tied to the lifecycle of this result.
smartCards	Map<Int, SmartCard>	—	3.0	All <u>SmartCard</u> instances corresponding to successful selection cases, indexed by the selectionCaseId supplied to prepareSelectionCase . Possibly empty.
activeSelectionCaseId	Int?	—	3.0	Identifier of the last successful selection case in execution order, whose smart card is the only one remaining active, or null if no selection case succeeded. When non- null , it is a key of smartCards .

5.2.7. Single Card Selection Result

Name	SingleCardSelectionResult
Kind	Data
Namespace	reader.selection
Since	3.0
Purpose	Result of a selection scenario executed by a <u>SingleCardSelectionManager</u> .

The execution stops at the first successful selection case, so the result exposes at most one smart card, identified by the **selectionCaseId** supplied to **prepareSelectionCase**. **selectionCaseId** and **smartCard** are either both **null**, when no selection case succeeded, or both non-**null**.

Name	Type	Default	Since	Description
cardType	CardType (CNA-TRD-API)	—	3.0	Type of the detected card, as defined by the CardType enumeration of the Terminal Reader Definitions API (see CNA-TRD-API). UNKNOWN when the card type could not be identified. The value is intrinsically tied to the lifecycle of this result.
selectionCaseId	Int?	—	3.0	Identifier of the successful selection case, or null if no selection case succeeded.
smartCard	<u>SmartCard</u> ?	—	3.0	The active <u>SmartCard</u> produced by the successful selection case, or null if no selection case succeeded.

5.3. API Interfaces

5.3.1. Card Reader

Name	CardReader
Kind	Interface
Namespace	reader
Since	1.0
Purpose	Represents a card reader capable of driving the underlying hardware to manage card detection.

Get Name

Signature	getName() → String
Since	1.0
Description	Returns the name of the reader.
Returns	A non-empty String identifying the reader. The value is stable for the lifetime of the reader instance.
Pre-conditions	None.
Errors	None.

Is Card Present

Signature	isCardPresent() → Boolean
Since	1.0
Description	Indicates whether a card is currently detected by the reader. What counts as present depends on the reader's communication mode (see isContactless): on a contact reader, a card is present when it is physically inserted in the slot; on a contactless reader, a card is present when it lies within range of the antenna (in the RF field).
Returns	true if a card is present—inserted for a contact reader, or within the field for a contactless reader— false otherwise.
Pre-conditions	None.
Errors	ReaderCommunication —if the communication with the reader has failed.

Is Contactless

Signature	isContactless() → Boolean
Since	1.0
Description	Indicates whether the card communication mode is contactless.
Returns	true if the communication mode is contactless, false otherwise.
Pre-conditions	None.
Errors	None.

5.3.2. Card Reader Provider

Name	CardReaderProvider
Kind	Interface

Namespace	reader
Since	3.0
Purpose	Standardised discovery and access surface for the card readers available in the execution environment.

The `CardReaderProvider` is obtained from the `ReaderApiFactory`. It offers an implementation-agnostic view over the readers currently registered in the environment. Reader lifecycle (registration and un-registration) is handled by the runtime environment; the provider is a **read-only** view reflecting the readers active at the moment of each call.

Find Reader

Signature	<code>findReader(readerNameRegex: String) → CardReader?</code>
Since	3.0
Description	Returns the first reader whose name matches the supplied regular expression. This lookup is convenient when reader names embed variable elements (serial numbers, USB port index, etc.) that the application cannot hard-code.
Parameters	<code>readerNameRegex</code> —the regular expression matched against the reader names.
Returns	A <code>CardReader</code> whose name matches the expression, or <code>null</code> if none matches.
Pre-conditions	Argument —every parameter is non- <code>null</code> . Argument —the supplied string is a valid regular expression.
Errors	None.

Get Reader

Signature	<code>getReader(readerName: String) → CardReader?</code>
Since	3.0
Description	Returns the reader whose name exactly matches the supplied string.
Parameters	<code>readerName</code> —the exact name of the reader to retrieve.
Returns	A <code>CardReader</code> whose name equals <code>readerName</code> , or <code>null</code> if no such reader is registered.
Pre-conditions	Argument —every parameter is non- <code>null</code> .
Errors	None.

Get Reader Names

Signature	<code>getReaderNames() → Set<String></code>
Since	3.0
Description	Returns the names of all readers currently registered in the execution environment.
Returns	A <code>Set</code> of reader names; empty if no reader is registered.
Pre-conditions	None.
Errors	None.

Get Readers

Signature	<code>getReaders() → Set<CardReader></code>
Since	3.0
Description	Returns all readers currently registered in the execution environment.

Returns	A Set of CardReader references; empty if no reader is registered.
Pre-conditions	None.
Errors	None.

5.3.3. Card Selection Manager

Name	CardSelectionManager
Kind	Interface
Namespace	reader.selection
Since	1.0
Purpose	Common contract of the card selection managers: preparation of the card selection scenario and export/import of the scenario and of its processed result. An instance is obtained through one of the specialised managers created by the ReaderApiFactory .

A **card selection scenario** is composed of one or more **selection cases**, each identified by an application-chosen identifier and backed by a [CardSelectionExtension](#). A selection case targets a specific card and MAY define additional commands to be executed after the successful selection. Selection cases are executed in the order in which they were prepared, regardless of their identifiers. When a selection case fails, the manager tries the next selection case defined in the scenario, until no further selection case is available. All the selection cases of a scenario target the **same card** presented to the reader. What a scenario discriminates is, in practice, the **applications** hosted by that card—for example the Calypso applications of a multi-application card. The qualifiers **single**, **sequential** and **multichannel** in the names of the managers therefore describe how the scenario is executed and how many selections are retained, never a number of cards. The term **card** is kept throughout the API because a selection case also targets cards that host no application, such as storage cards, where the selection targets the card itself.

The selection mode is not a parameter of the execution: it is carried by the type of the manager, each specialised manager being obtained from its own factory operation and producing its own result type.

Manager	Behaviour	Execution	Result
SingleCardSelectionManager	Single-channel. Stops at the first successful selection case.	Explicit or scheduled	SingleCardSelectionResult
SequentialCardSelectionManager	Single-channel. Processes every selection case regardless of intermediate successes.	Explicit or scheduled	SequentialCardSelectionResult
MultichannelCardSelectionManager	ISO/IEC 7816-4 multi-channel cards. Places each successful selection case on its own logical channel, according to a ChannelSelectionPolicy .	Explicit	MultichannelCardSelectionResult

This interface carries the operations common to the three modes. The operations whose result depends on the mode—execution, scheduling, parsing of a scheduled response and import of a processed scenario—are defined by each specialised manager. See [Card selection modes](#) for an end-to-end view of the synchronous and asynchronous flows.

A scenario is considered **prepared** once at least one selection case has been appended via `prepareSelectionCase` or imported via `importCardSelectionScenario`. A prepared scenario MAY be executed multiple times; each execution operates on the scenario as it stood at the time of the call.

Export Card Selection Scenario

Signature	<code>exportCardSelectionScenario() → String</code>
Since	1.1

Description	Exports the content of the current prepared selection scenario, including the identifiers of its selection cases, as a String . The result MAY be imported into any card selection manager, whatever its type, via <code>importCardSelectionScenario</code> .
Returns	A non-empty String .
Pre-conditions	None.
Errors	None.
See also	<code>importCardSelectionScenario</code>

Export Processed Card Selection Scenario

Signature	<code>exportProcessedCardSelectionScenario() → String</code>
Since	1.3
Description	Exports the content of the previously processed selection scenario as a String . The result MAY be imported through the <code>importProcessedCardSelectionScenario</code> operation of a manager of the same type. Caution: if the local environment does not have all the card extensions involved in the scenario, the selection result produced locally will not contain any active selection. In that case, the processed scenario MUST be exported so that it can be re-imported and interpreted on a manager which has the relevant card extensions.
Returns	A non-empty String .
Pre-conditions	State —the scenario has been processed successfully, via <code>processCardSelectionScenario</code> or <code>parseScheduledCardSelectionsResponse</code> .
Errors	None.
See also	<code>SingleCardSelectionManager.importProcessedCardSelectionScenario</code> <code>SequentialCardSelectionManager.importProcessedCardSelectionScenario</code> <code>MultichannelCardSelectionManager.importProcessedCardSelectionScenario</code>

Import Card Selection Scenario

Signature	<code>importCardSelectionScenario(cardSelectionScenario: String) → Self</code>
Since	1.1
Description	Replaces the current scenario with a previously exported scenario, including the identifiers of its selection cases. The selection cases prepared or imported before the call are discarded.
Parameters	cardSelectionScenario —the String containing the exported card selection scenario.
Returns	The current instance (Self).
Pre-conditions	Argument —every parameter is non- null . Argument —the string has been produced by <code>exportCardSelectionScenario</code> and is well-formed.
Errors	None.
See also	<code>exportCardSelectionScenario</code>

Prepare Selection Case

Signature	<code>prepareSelectionCase(selectionCaseId: Int, cardSelector: CardSelector, cardSelectionExtension: CardSelectionExtension) → Self</code>
Since	2.0

Description	Appends a selection case, identified by <code>selectionCaseId</code> , to the scenario. Selection cases are executed in the order in which they were prepared, regardless of their identifiers. The identifier is the key under which the corresponding <code>SmartCard</code> is exposed in the selection result.
Parameters	<code>selectionCaseId</code> —the identifier of the selection case, chosen by the application. <code>cardSelector</code> (<code>CardSelector</code>)—the card selector containing the filters to be used to select the card. <code>cardSelectionExtension</code> (<code>CardSelectionExtension</code>)—the card selection extension to be used to parse the card selection response.
Returns	The current instance (<code>Self</code>).
Pre-conditions	Argument —every parameter is non- <code>null</code> . Argument — <code>selectionCaseId</code> is not already used by a selection case of the scenario.
Errors	None.

5.3.4. Card Selector

Name	<code>CardSelector</code>
Kind	Sealed interface
Namespace	<code>reader.selection</code>
Since	2.0
Purpose	Common supertype of the card selectors. Carries no member; it exists to let an operation accept any selector.

Its permitted implementations are `BasicCardSelector` and `IsoCardSelector`, which carry the filters restricting the selection process.

5.3.5. Multichannel Card Selection Manager

Name	<code>MultichannelCardSelectionManager</code>
Kind	Interface
Namespace	<code>reader.selection</code>
Extends	<code>CardSelectionManager</code>
Since	3.0
Purpose	Card selection manager executing every selection case of the scenario, keeping each matching application of an ISO/IEC 7816-4 multi-channel card active simultaneously, each on its own logical channel. Out of scope for storage cards. An instance is obtained via <code>ReaderApiFactory.createMultichannelCardSelectionManager</code> .

Each successful selection case is placed on its own logical channel of the ISO/IEC 7816-4 card, the use of the basic channel being governed by the `ChannelSelectionPolicy` passed at execution time. All the smart cards of the resulting `MultichannelCardSelectionResult` remain active in parallel. Only the explicit execution is available: scheduled selection is not supported in multi-channel mode.

Import Processed Card Selection Scenario

Signature	<code>importProcessedCardSelectionScenario(processedCardSelectionScenario: String) → MultichannelCardSelectionResult</code>
Since	3.0
Description	Imports a previously exported processed selection scenario and returns the corresponding result.
Parameters	<code>processedCardSelectionScenario</code> —the <code>String</code> containing the exported processed card selection scenario.

Returns	A <u>MultichannelCardSelectionResult</u> .
Pre-conditions	Argument—every parameter is non-null. Argument—the string has been produced by <code>exportProcessedCardSelectionScenario</code> of a <code>MultichannelCardSelectionManager</code>, is well-formed and contains no more selection cases than the current scenario. State—the manager has been configured with the same selection scenario as the manager that produced the export. Capability—the local environment possesses every card extension involved in the scenario.
Errors	<u>InvalidCardResponse</u> —if the data returned by the card could not be interpreted.
See also	<u>exportProcessedCardSelectionScenario</u>

Process Card Selection Scenario

Signature	<pre>processCardSelectionScenario(reader: CardReader, channelSelectionPolicy: ChannelSelectionPolicy) → MultichannelCardSelectionResult</pre>
Since	3.0
Description	Explicitly executes the previously prepared selection scenario in multi-channel mode against the provided reader, placing each successful selection case on its own logical channel according to the requested channel policy, and returns the result. If the presented card is not a multi-channel ISO/IEC 7816-4 card, an <code>InvalidCardResponse</code> is raised at scenario execution time.
Parameters	<code>reader</code> (<u>CardReader</u>)—the reader through which the card is reached. <code>channelSelectionPolicy</code> (<u>ChannelSelectionPolicy</u>)—the policy governing the channels on which selections are placed.
Returns	A <u>MultichannelCardSelectionResult</u> .
Pre-conditions	Argument —every parameter is non-null.
Errors	<u>ReaderCommunication</u>—if communication with the reader has failed. <u>CardCommunication</u>—if communication with the card has failed. <u>InvalidCardResponse</u>—if the card does not support multi-channel or returned invalid data.

5.3.6. Observable Card Reader

Name	ObservableCardReader
Kind	Interface
Namespace	reader
Extends	<u>CardReader</u>
Since	1.0
Purpose	A <code>CardReader</code> capable of observing the insertion and removal of cards.

The observation contract is co-located with the start of detection: a single call to `startCardDetection` provides both the CardDetectionSettings that govern the polling behaviour and the CardReaderEventHandler that receives the resulting events.

The reader's complete insertion/processing/removal lifecycle—including when each observation event is produced—is described in [Reader observation lifecycle](#).

Clear Scheduled Card Selection Scenario

Signature	<code>clearScheduledCardSelectionScenario() → Unit</code>
Since	3.0
Description	Removes the card selection scenario scheduled on this reader, if any. From the next card insertion onwards, no scenario is executed and the <code>scheduledCardSelectionsResponse</code> of the resulting <code>CardReaderEvent</code> is <code>null</code>. The processing of a card already inserted is not affected. This operation is idempotent: calling it when no scenario is scheduled has no effect.
Pre-conditions	None.
Errors	None.
See also	SingleCardSelectionManager.scheduleCardSelectionScenario SequentialCardSelectionManager.scheduleCardSelectionScenario

End Card Processing

Signature	<code>endCardProcessing() → Unit</code>
Since	3.0
Description	Notifies the reader that the business processing of the current card has been completed. Upon receiving this notification, the reader: - performs the DESELECT sequence on the card (when applicable to the underlying technology) so that the card is brought to a clean idle state; - transitions to the waiting-for-card-removal phase, during which the observation cycle waits for the card to leave the reader's field before becoming able to detect a new card. In addition, the reader releases the references to the <code>SmartCard</code> instances held following the last selection (see the SmartCard lifecycle). This operation is idempotent: calling it twice on the same card processing cycle has no effect after the first call. It is RECOMMENDED to call <code>endCardProcessing</code> systematically (e.g. inside a finally block) at the end of a card processing whatever the outcome.
Pre-conditions	None.
Errors	None.

Start Card Detection

Signature	<pre>startCardDetection(settings: CardDetectionSettings, eventHandler: CardReaderEventHandler) → Unit</pre>
Since	3.0
Description	Activates card detection. Once activated, the application is notified of card events through the registered <code>CardReaderEventHandler</code>. The <code>CardDetectionSettings</code> configure the detection mode, the targeted RF technologies and, when applicable, the ECP frame to emit at polling startup. Parameters carried by <code>CardDetectionSettings</code> that are not supported by the underlying reader (for example an unsupported <code>RfTechnology</code>, or an ECP frame on a reader that does not support ECP) are silently ignored. A <code>WARN</code>-level message is typically logged in such cases. No error is raised.
Parameters	<code>settings</code> (<code>CardDetectionSettings</code>) — the detection configuration. <code>eventHandler</code> (<code>CardReaderEventHandler</code>) — the application-side handler that receives reader events and errors.
Pre-conditions	Argument —every parameter is non- <code>null</code> .

Errors	None.
--------	-------

Stop Card Detection

Signature	stopCardDetection() → Unit
Since	1.0
Description	Stops card detection. After this operation has returned, the registered event handler does not receive card events until detection is restarted.
Pre-conditions	None.
Errors	None.

5.3.7. Reader API Factory

Name	ReaderApiFactory
Kind	Interface
Namespace	reader
Since	2.0
Purpose	Factory used by the application to obtain instances of the public types provided by the API.

The **ReaderApiFactory** is the single entry point through which the application instantiates the public types of the API. It is provided as a single instance. Its **create** operations each return a fresh, fully initialised object, whereas **getCardReaderProvider** gives access to the **CardReaderProvider** of the execution environment.

Create Multichannel Card Selection Manager

Signature	createMultichannelCardSelectionManager() → MultichannelCardSelectionManager
Since	3.0
Description	Creates a <u>MultichannelCardSelectionManager</u> , the service used to prepare and execute a card selection scenario on a multi-channel ISO/IEC 7816-4 card.
Returns	A <u>MultichannelCardSelectionManager</u> .
Pre-conditions	None.
Errors	None.

Create Sequential Card Selection Manager

Signature	createSequentialCardSelectionManager() → SequentialCardSelectionManager
Since	3.0
Description	Creates a <u>SequentialCardSelectionManager</u> , the service used to prepare and execute a card selection scenario processing every selection case.
Returns	A <u>SequentialCardSelectionManager</u> .
Pre-conditions	None.
Errors	None.

Create Single Card Selection Manager

Signature	createSingleCardSelectionManager() → SingleCardSelectionManager
Since	3.0

Description	Creates a <u><a>SingleCardSelectionManager</u> , the service used to prepare and execute a card selection scenario stopping at the first successful selection case.
Returns	A <u><a>SingleCardSelectionManager</u> .
Pre-conditions	None.
Errors	None.

Get Card Reader Provider

Signature	<code>getCardReaderProvider() → CardReaderProvider</code>
Since	3.0
Description	Returns the <u><a>CardReaderProvider</u> of the execution environment, giving standardised access to the readers available in it. Successive calls return the same instance.
Returns	A <u><a>CardReaderProvider</u> .
Pre-conditions	None.
Errors	None.

5.3.8. Scheduled Card Selections Response

Name	<code>ScheduledCardSelectionsResponse</code>
Kind	Marker interface
Namespace	<code>reader.selection</code>
Since	1.0
Purpose	Carrier for the response of the execution of a scheduled selection scenario, provided by a <u><a>CardReaderEvent</u> .

This interface declares no operation. It conveys the card responses to one or more of the scenario's selection cases (selection step itself plus any subsequent commands). Its content **MUST** be interpreted through the `parseScheduledCardSelectionsResponse` operation of the manager that scheduled the scenario (SingleCardSelectionManager or SequentialCardSelectionManager).

5.3.9. Sequential Card Selection Manager

Name	<code>SequentialCardSelectionManager</code>
Kind	Interface
Namespace	<code>reader.selection</code>
Extends	<u><a>CardSelectionManager</u>
Since	3.0
Purpose	Card selection manager executing every selection case of the scenario, selecting in turn each matching application of the card; only the last selected application remains active. Intended for cards hosting several applications, and therefore not applicable to storage cards. An instance is obtained via <u><a>ReaderApiFactory.createSequentialCardSelectionManager</u> .

Every selection case is tried, in the order in which they were prepared, whatever the outcome of the previous ones. Since the card is operated on a single channel, each successful selection deactivates the smart card produced by the previous one: in the resulting SequentialCardSelectionResult, only the smart card of the last successful selection case remains active. To keep several selections active in parallel on a multi-channel ISO/IEC 7816-4 card, the MultichannelCardSelectionManager **MUST** be used instead. The scenario can be executed explicitly, on a card already present, or scheduled on an ObservableCardReader.

Import Processed Card Selection Scenario

Signature	<code>importProcessedCardSelectionScenario(processedCardSelectionScenario: String) → SequentialCardSelectionResult</code>
Since	3.0
Description	Imports a previously exported processed selection scenario and returns the corresponding result.
Parameters	processedCardSelectionScenario —the String containing the exported processed card selection scenario.
Returns	A <u>SequentialCardSelectionResult</u> .
Pre-conditions	Argument —every parameter is non-null. Argument —the string has been produced by <code>exportProcessedCardSelectionScenario</code> of a <code>SequentialCardSelectionManager</code> , is well-formed and contains no more selection cases than the current scenario. State —the manager has been configured with the same selection scenario as the manager that produced the export. Capability —the local environment possesses every card extension involved in the scenario.
Errors	<u>InvalidCardResponse</u> —if the data returned by the card could not be interpreted.
See also	<u>exportProcessedCardSelectionScenario</u>

Parse Scheduled Card Selections Response

Signature	<code>parseScheduledCardSelectionsResponse(scheduledCardSelectionsResponse: ScheduledCardSelectionsResponse) → SequentialCardSelectionResult</code>
Since	3.0
Description	Analyses the responses provided by a <u>CardReaderEvent</u> following the insertion of a card and the execution of the selection scenario scheduled with <code>scheduleCardSelectionScenario</code> .
Parameters	scheduledCardSelectionsResponse (<u>ScheduledCardSelectionsResponse</u>)—the card selection scenario execution response.
Returns	A <u>SequentialCardSelectionResult</u> .
Pre-conditions	Argument —every parameter is non-null.
Errors	<u>InvalidCardResponse</u> —if the data returned by the card could not be interpreted.

Process Card Selection Scenario

Signature	<code>processCardSelectionScenario(reader: CardReader) → SequentialCardSelectionResult</code>
Since	3.0
Description	Explicitly executes the previously prepared selection scenario against the provided reader, processing every selection case regardless of intermediate successes, and returns the result.
Parameters	reader (<u>CardReader</u>)—the reader through which the card is reached.
Returns	A <u>SequentialCardSelectionResult</u> .
Pre-conditions	Argument —every parameter is non-null.
Errors	<u>ReaderCommunication</u> —if communication with the reader has failed. <u>CardCommunication</u> —if communication with the card has failed. <u>InvalidCardResponse</u> —if the card returned invalid data during the selection process, or if the status word check is enabled in the card request and the card returned an unexpected code.

Schedule Card Selection Scenario

Signature	<code>scheduleCardSelectionScenario(observableCardReader: ObservableCardReader, cardPresenceNotificationPolicy: CardPresenceNotificationPolicy) → Unit</code>
Since	3.0
Description	Schedules the execution of the prepared selection scenario as soon as a card is presented to the supplied observable reader. Events are pushed to the registered event handler according to the requested notification policy. The result of the execution MUST be parsed via parseScheduledCardSelectionsResponse .
Parameters	<code>observableCardReader</code> (ObservableCardReader)—the reader through which the card is reached. <code>cardPresenceNotificationPolicy</code> (CardPresenceNotificationPolicy)—the desired card-presence notification policy.
Pre-conditions	Argument —every parameter is non-null.
Errors	None.

5.3.10. Single Card Selection Manager

Name	<code>SingleCardSelectionManager</code>
Kind	Interface
Namespace	<code>reader.selection</code>
Extends	CardSelectionManager
Since	3.0
Purpose	Card selection manager executing the scenario up to the first matching selection case: the first matching card application—for example the Calypso application of a Calypso card—or the card itself when it hosts no application, as a storage card. An instance is obtained via ReaderApiFactory.createSingleCardSelectionManager .

The selection cases are tried in the order in which they were prepared; the execution stops as soon as one of them succeeds. The resulting [SingleCardSelectionResult](#) therefore exposes at most one smart card. The scenario can be executed explicitly, on a card already present, or scheduled on an [ObservableCardReader](#).

Import Processed Card Selection Scenario

Signature	<code>importProcessedCardSelectionScenario(processedCardSelectionScenario: String) → SingleCardSelectionResult</code>
Since	3.0
Description	Imports a previously exported processed selection scenario and returns the corresponding result.
Parameters	<code>processedCardSelectionScenario</code> —the <code>String</code> containing the exported processed card selection scenario.
Returns	A SingleCardSelectionResult .
Pre-conditions	Argument —every parameter is non-null. Argument —the string has been produced by <code>exportProcessedCardSelectionScenario</code> of a SingleCardSelectionManager , is well-formed and contains no more selection cases than the current scenario. State —the manager has been configured with the same selection scenario as the manager that produced the export. Capability —the local environment possesses every card extension involved in the scenario.
Errors	InvalidCardResponse —if the data returned by the card could not be interpreted.

See also [exportProcessedCardSelectionScenario](#)

Parse Scheduled Card Selections Response

Signature	<code>parseScheduledCardSelectionsResponse(scheduledCardSelectionsResponse: ScheduledCardSelectionsResponse) → SingleCardSelectionResult</code>
Since	3.0
Description	Analyses the responses provided by a CardReaderEvent following the insertion of a card and the execution of the selection scenario scheduled with <code>scheduleCardSelectionScenario</code> .
Parameters	<code>scheduledCardSelectionsResponse</code> (ScheduledCardSelectionsResponse) —the card selection scenario execution response.
Returns	A SingleCardSelectionResult .
Pre-conditions	Argument —every parameter is non- <code>null</code> .
Errors	InvalidCardResponse —if the data returned by the card could not be interpreted.

Process Card Selection Scenario

Signature	<code>processCardSelectionScenario(reader: CardReader) → SingleCardSelectionResult</code>
Since	3.0
Description	Explicitly executes the previously prepared selection scenario against the provided reader, stopping at the first successful selection case, and returns the result.
Parameters	<code>reader</code> (CardReader) —the reader through which the card is reached.
Returns	A SingleCardSelectionResult .
Pre-conditions	Argument —every parameter is non- <code>null</code> .
Errors	ReaderCommunication —if communication with the reader has failed. CardCommunication —if communication with the card has failed. InvalidCardResponse —if the card returned invalid data during the selection process, or if the status word check is enabled in the card request and the card returned an unexpected code.

Schedule Card Selection Scenario

Signature	<code>scheduleCardSelectionScenario(observableCardReader: ObservableCardReader, cardPresenceNotificationPolicy: CardPresenceNotificationPolicy) → Unit</code>
Since	3.0
Description	Schedules the execution of the prepared selection scenario as soon as a card is presented to the supplied observable reader. Events are pushed to the registered event handler according to the requested notification policy. The result of the execution MUST be parsed via parseScheduledCardSelectionsResponse .
Parameters	<code>observableCardReader</code> (ObservableCardReader) —the reader through which the card is reached. <code>cardPresenceNotificationPolicy</code> (CardPresenceNotificationPolicy) —the desired card-presence notification policy.
Pre-conditions	Argument —every parameter is non- <code>null</code> .
Errors	None.

5.4. SPI Interfaces

5.4.1. Card Reader Event Handler

Name	CardReaderEventHandler
Kind	Interface (SPI)
Namespace	reader.spi
Since	3.0
Purpose	Interface that the application MUST implement to receive reader events and observation errors from an <u>ObservableCardReader</u> .

This SPI unifies the reception of nominal events (`onReaderEvent`) and of fatal observation errors (`onReaderError`) into a single contract, which the application implements once and registers at the call to startCardDetection.

On Reader Error

Signature	<pre>onReaderError(context: String, readerName: String, error: Any) → Unit</pre>
Since	3.0
Description	Called when a fatal error occurs on the observed reader. After the operation returns, the observation process is considered stopped.
Parameters	<code>context</code> — the context describing the operation that failed. <code>readerName</code> — the name of the reader on which the error occurred. <code>error</code> — the original error that triggered the notification.
Pre-conditions	Argument — every parameter is non- <code>null</code> .
Errors	None.

On Reader Event

Signature	<code>onReaderEvent(cardReaderEvent: CardReaderEvent) → Unit</code>
Since	3.0
Description	Called when a reader event occurs. Dispatch is sequential and synchronous unless a different policy is explicitly documented.
Parameters	<code>cardReaderEvent</code> (<u>CardReaderEvent</u>) — the event payload.
Pre-conditions	Argument — every parameter is non- <code>null</code> .
Errors	None. Any error raised by the handler is routed to <code>onReaderError</code> .

5.4.2. Card Selection Extension

Name	CardSelectionExtension
Kind	Marker interface (SPI)
Namespace	reader.selection.spi
Since	2.0

Purpose	Provided by card extensions to enrich a selection case with additional commands and to interpret the response in order to build the corresponding <u>SmartCard</u> .
----------------	--

This interface declares no operation at the API level; the contract between the selection manager and the extension is defined by the card extension specification consuming this SPI.

5.4.3. Card Transaction Manager

Name	CardTransactionManager
Kind	Interface (SPI)
Namespace	reader.transaction.spi
Since	2.1
Purpose	Root contract common to every card transaction manager exposed by a card extension. Defines the single operation <code>processCommands</code> .

To exchange data with a card, the application first prepares the commands to transmit (via the operations of the consuming card extension) and then processes them. The preparation step allows commands to be grouped to minimise network exchanges, which is especially valuable in distributed architectures. The SmartCard registered with the manager is updated after each data exchange with the card.

Process Commands

Signature	<code>processCommands() → Unit</code>
Since	3.0
Description	Processes all previously prepared commands. All APDUs corresponding to the prepared commands are sent to the card, their responses retrieved and used to update the <u>SmartCard</u> associated with the transaction. For write commands, the <u>SmartCard</u> is updated only when the command is successful. The process is interrupted at the first failed command. The underlying logical channel is left open after processing.
Pre-conditions	None.
Errors	<u>ReaderCommunication</u> —if a communication error with the reader occurs. <u>CardCommunication</u> —if a communication error with the card occurs. <u>InvalidCardResponse</u> —if a command returns an unexpected status, or if the manager was obtained through <code>asMultichannelCardTransactionManager</code> and the card does not support multi-channel.

5.4.4. ISO Card Transaction Manager

Name	IsoCardTransactionManager
Kind	Interface (SPI)
Namespace	reader.transaction.spi
Extends	<u>CardTransactionManager</u>
Since	3.0
Purpose	Intermediate contract for ISO/IEC 7816-4 cards. Exposes the on-demand cast to multi-channel.

As Multichannel Card Transaction Manager

Signature	<code>asMultichannelCardTransactionManager() → MultichannelCardTransactionManager</code>
Since	3.0

Description	Returns a view of this manager as a <u>MultichannelCardTransactionManager</u> , granting access to the multi-channel operations. The conversion itself never fails. If the underlying ISO/IEC 7816-4 card does not support multi-channel operation, an InvalidCardResponse is raised only when the commands are processed through the returned manager.
Returns	A <u>MultichannelCardTransactionManager</u> .
Pre-conditions	None.
Errors	None.

5.4.5. ISO Smart Card

Name	IsoSmartCard
Kind	Interface (SPI)
Namespace	<code>reader.selection.spi</code>
Extends	<u>SmartCard</u>
Since	2.0
Purpose	ISO/IEC 7816-4 smart card whose communication has been established after a successful selection.

In addition to the inherited power-on data, an **IsoSmartCard** exposes the response to the **Select Application** command. Both the power-on data and the **Select Application** response are optional, but at least one of them is always present.

Get Select Application Response

Signature	<code>getSelectApplicationResponse() → ByteArray?</code>
Since	1.0
Description	Returns the data received from the card in response to the Select Application command, including the status word.
Returns	The non-empty response bytes, or null if no Select Application command has been performed.
Pre-conditions	None.
Errors	None.

Is Basic Channel

Signature	<code>isBasicChannel() → Boolean</code>
Since	3.0
Description	Indicates whether this smart card is attached to the basic channel (channel 0) of the underlying ISO/IEC 7816-4 card, or to an additional logical channel.
Returns	true if attached to the basic channel, false if attached to an additional logical channel.
Pre-conditions	None.
Errors	None.

5.4.6. Multichannel Card Transaction Manager

Name	MultichannelCardTransactionManager
Kind	Interface (SPI)

Namespace	reader.transaction.spi
Extends	<u>CardTransactionManager</u>
Since	3.0
Purpose	Multi-channel contract. Exposes the operations to close a logical channel explicitly.

This contract MAY be obtained either:

- through IsoCardTransactionManager.asMultichannelCardTransactionManager, for ISO/IEC 7816-4 cards on which multi-channel is an optional capability;
- directly as the declared supertype of the transaction manager of a card extension targeting intrinsically multi-channel cards.

Close Channel

Signature	closeChannel() → Unit
Since	3.0
Description	Closes the underlying logical channel without processing any pending command. This operation is idempotent: calling it on an already-closed channel has no effect.
Pre-conditions	None.
Errors	<u>ReaderCommunication</u> —if a communication error with the reader occurs. <u>CardCommunication</u> —if a communication error with the card occurs.

Process Commands And Close Channel

Signature	processCommandsAndCloseChannel() → Unit
Since	3.0
Description	Processes all previously prepared commands and, upon success, closes the underlying logical channel. If any command fails, the channel is left open, as with processCommands .
Pre-conditions	None.
Errors	<u>ReaderCommunication</u> —if a communication error with the reader occurs. <u>CardCommunication</u> —if a communication error with the card occurs. <u>InvalidCardResponse</u> —if a command returns an unexpected status, or if the card does not support multi-channel.

5.4.7. Smart Card

Name	SmartCard
Kind	Interface (SPI)
Namespace	reader.selection.spi
Since	1.0
Purpose	Basic smart card whose communication has been established after a successful selection. The instance is ready to receive APDUs.

Card extensions implement (and may extend) this interface to expose the data they collected during the selection.

Get Power On Data

Signature	getPowerOnData() → String?
-----------	----------------------------

Since	1.0
Description	Returns the card's power-on data, as collected by the reader at the moment the card was inserted. - For a contact reader, this is the Answer To Reset (ATR) defined by ISO/IEC 7816. - For a contactless reader, the reader decides what this data is. Some contactless readers provide a virtual ATR (partially standardised by the PC/SC standard), while other devices may have their own definition, including for example elements from the anti-collision stage of ISO/IEC 14443 (ATQA, ATQB, ATS, SAK, etc.) or any proprietary definitions. Because this data varies from one reader to another, it is exposed here as a String that MAY be either a hexadecimal string or any other relevant representation.
Returns	The non-empty power-on data, or null if no power-on data is available.
Pre-conditions	None.
Errors	None.

Is Active

Signature	<code>isActive() → Boolean</code>
Since	3.0
Description	Returns whether this smart card is currently active on its logical channel. A smart card is considered active between its creation (as the result of a successful selection case) and the occurrence of any of the following events, at which point the reader deactivates it (<code>isActive()</code> returns false) and releases its reference: - a new selection in single-channel mode (the previous smart cards become obsolete); - an explicit channel closure request through <code>MultichannelCardTransactionManager.closeChannel</code> or <code>processCommandsAndCloseChannel</code>; - a call to <code>ObservableCardReader.endCardProcessing</code>; - an error indicating that the card is no longer contactable (<code>CardCommunication</code> or <code>ReaderCommunication</code>). Until then, the reader holds the references to the SmartCard instances produced by the last successful selection. Once a smart card has been deactivated, any further attempt to use it is detected prior to any actual exchange with the card.
Returns	true if the smart card is active, false otherwise.
Pre-conditions	None.
Errors	None.

5.5. Enumerations

5.5.1. Card Presence Notification Policy

Name	<code>CardPresenceNotificationPolicy</code>
Kind	Enumeration
Namespace	<code>reader.selection</code>
Since	3.0
Purpose	Options applied when a card is detected, used to decide which cards trigger event-handler notifications.

In the reader observation lifecycle, this policy determines whether a detected card is treated as processable—see [Reader observation lifecycle](#).

Value	Since	Description
ALWAYS	3.0	All cards presented to the reader are notified, regardless of the result of the selection.
MATCHED_ONLY	3.0	Only cards that have been successfully selected are notified. The others are ignored.

5.5.2. Card Reader Event Type

Name	CardReaderEventType
Kind	Enumeration
Namespace	reader
Since	1.0
Purpose	Possible card reader events.

Value	Since	Description
CARD_INSERTED	1.0	A card has been inserted, with or without a specific selection.
CARD_MATCHED	1.0	A card has been inserted that matches the selection criteria.
CARD_REMOVED	1.0	The card has been removed from the reader.
READER_UNREGISTERED	3.0	The reader has been unregistered and is no longer usable.

5.5.3. Channel Selection Policy

Name	ChannelSelectionPolicy
Kind	Enumeration
Namespace	reader.selection
Since	3.0
Purpose	Policy governing the use of the basic channel in a multi-channel selection scenario.

Value	Since	Description
ALLOW_BASIC_CHANNEL	3.0	The basic channel (channel 0) MAY host selection cases in addition to the additional logical channels.
LOGICAL_CHANNEL_ONLY	3.0	Selection cases are placed only on additional logical channels; the basic channel is not used.

5.5.4. Detection Mode

Name	DetectionMode
Kind	Enumeration
Namespace	reader
Since	3.0
Purpose	Defines the behaviour to apply after a card processing cycle has terminated.

In the reader observation lifecycle, this setting is the guard that decides whether detection resumes after a card-processing cycle—see [Reader observation lifecycle](#).

Value	Since	Description
REPEATING	3.0	The reader continues waiting for the next card insertion.
SINGLE_SHOT	3.0	The reader stops detection after the current card-processing cycle; a new call to startCardDetection is required to detect further cards.

5.5.5. File Control Information

Name	FileControlInformation
Kind	Enumeration
Namespace	reader.selection
Since	2.0
Purpose	Types of templates returned in response to the Select Application command, according to ISO/IEC 7816-4.

Value	Since	Description
FCI	2.0	File Control Information.
FCP	2.0	File Control Parameters.
FMD	2.0	File Management Data.
NO_RESPONSE	2.0	No response expected.

5.5.6. File Occurrence

Name	FileOccurrence
Kind	Enumeration
Namespace	reader.selection
Since	2.0
Purpose	Navigation options through the applications contained on the card according to ISO/IEC 7816-4.

Value	Since	Description
FIRST	2.0	First occurrence.
LAST	2.0	Last occurrence.
NEXT	2.0	Next occurrence.
PREVIOUS	2.0	Previous occurrence.

5.6. Errors

Every error defined below carries two properties: **message** (**String**), which describes the condition, and **cause** (**Any?**), the underlying error that triggered it, **null** when there is none. They are not restated for each error.

5.6.1. Card Communication

Name	CardCommunication
Kind	Error
Namespace	reader
Since	1.0

Purpose	Indicates that the communication with the card has failed, typically because the card was removed from the reader before the exchange could complete.
----------------	---

5.6.2. Invalid Card Response

Name	InvalidCardResponse
Kind	Error
Namespace	reader
Since	2.1
Purpose	Indicates that a response received from the card was invalid—typically when the returned status word does not match any of those expected—either during a selection process or during the processing of a transaction. This error is also raised when an ISO/IEC 7816-4 card does not support a requested multi-channel operation.

5.6.3. Reader Communication

Name	ReaderCommunication
Kind	Error
Namespace	reader
Since	1.0
Purpose	Indicates that the communication with the reader has failed. The most likely cause is a physical disconnection of the reader, but other technical problems MAY also be at the origin of the failure.